
title: “Práctico 2 — Resuelto paso a paso” subtitle:
“Manipulación de la representación interna de datos en programas — Arquitectura de Computadoras FING UdelaR”

Práctico 2 — Resuelto paso a paso

Nota: el material del curso trae código y ejemplos resueltos para las teóricas y los Ejercicios 1 a 6. Verifiqué cada uno trazando el código a mano con los valores de ejemplo del propio `main()`. Los Ejercicios 7, 8 y 9 **no tenían solución en el material del curso** — están resueltos acá desde cero, con ejemplos numéricos verificados (incluyendo casos conocidos como IEEE 754 de 1.0 y productos de punto flotante).

Preguntas teóricas

(a) Extraer bits con `&` y `>>`

1. Construí una **máscara**: una tira del mismo largo que la palabra, con 1's exactamente en las posiciones que querés extraer y 0's en el resto.
2. Hací `palabra & mascara` — el resultado tiene 0's en todos lados salvo una copia de los bits originales en las posiciones marcadas.
3. Si la posición más baja de los bits que extrajiste es n , desplazá el resultado n lugares a la derecha (`>> n`) para que quede alineado en la posición 0.

(b) Extraer bits con `/` y `%`

Sea n la posición inicial (más baja) del rango a extraer y m la posición final (más alta), contando desde 0 a la derecha:

1. `t' = t / 2^n` — descarta los bits por debajo de la posición n (equivalente a un shift right).
2. `t'' = t' % 2^k`, con $k = m - n + 1$ — se queda solo con los k bits que querías, descartando lo que esté por encima.

(c) `char` vs. `unsigned char`

Con 8 bits, `char` (con signo) representa -128 a 127 (el bit más significativo indica signo), mientras que `unsigned char` representa 0 a 255 (todos los bits cuentan como magnitud). El mismo patrón de bits se **interpreta distinto** según el tipo — esto importa especialmente en comparaciones (`if (c > 200)` nunca es cierto con `char` con signo) y en operaciones aritméticas con acarreo/desborde.

Metodología general — Promoción de tipos en C

Esta es la fuente más común de errores de “un bit” en este práctico. Regla de oro de C: en cualquier operación aritmética o bit a bit, los operandos de tipo `char` o `short` se **promueven primero a `int` (32 bits)**, respetando el signo (si el tipo era con signo, se extiende el bit de signo; si era `unsigned`, se rellena con ceros). La operación se hace en 32 bits, y **recién al final**, si el resultado se asigna a una variable más chica, se trunca a esa cantidad de bits (tomando los bits menos significativos).

Procedimiento para resolver estos ejercicios a mano:

1. Escribí cada operando en su representación de bits **original** (8 o 16 bits).
2. Promovelo a `int` de 32 bits, extendiendo el signo si corresponde.
3. Hacé la operación completa en 32 bits.
4. Truncá el resultado final a la cantidad de bits del tipo destino (tomando los bits menos significativos).

Ejercicio 1 — Trazar operaciones con promoción de tipos

A) `SHORT A=0XA87B; SHORT B=0X771C; SHORT C=A|B; CHAR D=C>>4;`

a = 1010 1000 0111 1011, b = 0111 0111 0001 1100

`c = a | b` (OR bit a bit, ambos ya de 16 bits, se opera y se trunca a short):

```
1010 1000 0111 1011
0111 0111 0001 1100
-----
1111 1111 0111 1111
```

c = 0xFF7F

`d = c >> 4`: c se promueve a int con **extensión de signo** (c es negativo, MSB=1) → 0xFFFFFFF7F. Shift aritmético derecho 4 lugares (rellena con el bit de signo, 1) → 0xFFFFFFF7. Se trunca a char (8 bits, byte menos significativo):

d = 0xF7 = 1111 0111 (= -9 en decimal)

B) `CHAR A=7; CHAR B=0XF3; CHAR C=5; SHORT D = A<<12 | B<<4 | C;`

$a=7$ (positivo) $\rightarrow$ int 0x00000007. $a<<12 = 0x00007000$.

$b=0xF3$ (char con signo = -13) $\rightarrow$ promovido con extensión de signo $\rightarrow$ int 0xFFFFFFFF3. $b<<4 = 0xFFFFFFFF30$.

$c=5 \rightarrow$ int 0x00000005.

OR de los tres: `0x00007000 | 0xFFFFFFFF30 | 0x00000005 = 0xFFFFFFFF35`

Truncado a short (16 bits menos significativos):

d = 0xFF35 = 1111 1111 0011 0101

C) `CHAR A=-3; CHAR B=0X80; SHORT RES = A+B;`

$a=-3 \rightarrow 0xFD$ (8 bits) $\rightarrow$ promovido con signo $\rightarrow$ int 0xFFFFFFFFD. $b=0x80$ (char con signo = -128) $\rightarrow$ promovido con signo $\rightarrow$ int 0xFFFFFFFF80.

`res = a+b` en 32 bits: 0xFFFFFFFFD + 0xFFFFFFFF80 = 0xFFFFFFFF7D (= -131 en decimal; verificación: $-3+(-128)=-131 \checkmark$)

Truncado a short:

res = 0xFF7D = 1111 1111 0111 1101

Ejercicio 2 — Suma BCD empaquetada

Metodología: separará cada byte en sus dos “dígitos” (nibble alto y bajo), sumá cada par de dígitos por separado, y si algún resultado parcial supera 9, sumale 6 (corrección BCD estándar) y propagá el acarreo al siguiente dígito.

```

short sumaBCD(char num1, char num2) {
    unsigned char decenas1 = (num1 & 0xF0) >> 4, unidades1 = num1 & 0x0F;
    unsigned char decenas2 = (num2 & 0xF0) >> 4, unidades2 = num2 & 0x0F;

    unsigned char unidades = unidades1 + unidades2;
    unsigned char decenas = decenas1 + decenas2;

    if (unidades > 9) { unidades += 6; decenas += 1; }
    if (decenas > 9) { decenas += 6; }

    return (decenas << 4) | unidades;
}

```

Ejemplo verificado: sumaBCD(0x25, 0x94) → dígitos: unidades=5+4=9 (≤ 9 , sin corrección), decenas=2+9=11 ($> 9 \rightarrow +6=17=0x11$, es decir “acarrea” un 1 a las centenas y deja 1 en decenas).

Resultado = $(0x11 \ll 4) | 9 = 0x119 \rightarrow$ dígitos BCD “1”, “1”, “9” = **119** (= 25+94 ✓, correcto incluso con 3 dígitos de resultado, porque el `short` de 16 bits tiene lugar de sobra para un nibble extra).

Ejercicio 3 — Conversión entre representaciones (16 bits)

A) COMPLEMENTO A 1 → VALOR ABSOLUTO Y SIGNO

```

short C1_a_VAyS(short num) {
    short S = (num & 0x8000) >> 15;
    short VA = (num & 0x7FFF);
    if (S == 1) VA = ~VA; // NOT de toda la palabra; los 15 bits bajos quedan con la
    magnitud
    return (S << 15) | (VA & 0x7FFF);
}

```

Por qué funciona: en C1, un número negativo es el NOT bit a bit de su valor positivo. Al aislar los 15 bits bajos e invertirlos, se recupera exactamente la magnitud.

Ejemplo verificado: num=0xFFFFC (C1). El signo es 1; NOT de toda la palabra 0xFFFFC = 0x0003 → magnitud=3. Resultado VA+S = 1 000000000000011 = **0x8003** (−3 en VA+S de 16 bits).

B) COMPLEMENTO A 2 → DESPLAZAMIENTO (D=128)

Metodología simplificada (más directa que ir por la magnitud): Desplazamiento(N) se calcula sumando directamente el sesgo D al **valor con signo N** — no hace falta extraer la magnitud por separado, porque N+D ya da el patrón de bits correcto (es la definición misma de exceso-D).

```
short C2_a_Desplazamiento(short num) {
    return num + 128;    // num ya está en complemento a 2 (valor con signo directo en C)
}
```

Ejemplo verificado: num=0xFFFFD = -3 (en C2). Resultado = -3+128 = **125 = 0000000001111101**.

Nota: el material del curso resuelve este mismo ejercicio con una rama adicional que primero calcula el opuesto y después lo vuelve a negar — es matemáticamente una no-operación ($\neg(\sim \text{num} + 1)$) es exactamente `num` otra vez, por la fórmula de complemento a 2), así que el resultado final coincide con esta versión más simple, pero sin el paso de más.

Ejercicio 4 — Extraer bits de una palabra

```
int extraerBits(int palabra, int bitAlto, int bitBajo) {
    int cantidadBits = bitAlto - bitBajo + 1;
    int mascara = (1 << cantidadBits) - 1;    // cantidadBits unos, en la posición 0
    return (palabra >> bitBajo) & mascara;
}
```

Metodología: primero desplazé la palabra para que el bit bajo deseado quede en la posición 0, después aplicé una máscara de exactamente *cantidadBits* unos para quedarte solo con esos bits.

Ejemplo del enunciado, verificado: palabra= `...000111001101`, bitAlto=6, bitBajo=2.

Bits 8 a 0 de la palabra: `1 1 1 0 0 1 1 0 1` (bit8...bit0). Los bits 6..2 son: bit6=1, bit5=0, bit4=0, bit3=1, bit2=1 → **10011**, exactamente lo que pide el enunciado. ✓

Máscara: cantidadBits=6-2+1=5 → mascara=(1<<5)-1=0b11111=31. `palabra>>2` alinea el bit2 en la posición 0, y el AND con 31 (5 unos) descarta todo lo que esté por encima del bit4 del resultado desplazado.

Ejercicio 5 — Paridad de una palabra de 32 bits

```
int paridad(int palabra) {
    int p = 0;
    for (int i = 0; i < 32; i++) {
        p ^= (palabra & 1);
        palabra >>= 1;
    }
    return p;    // 1 = cantidad impar de unos (bit de paridad PAR sería este mismo valor 'p')
}
```

Por qué funciona: el XOR acumulado de todos los bits da 1 si y solo si hay una cantidad **impar** de unos en total. Ese es exactamente el bit que hay que agregar para lograr **paridad par** (cantidad total de unos, dato+paridad, par). Para paridad impar, se usa el complemento: `!p`.

Ejemplo verificado: palabra=0b111 (tres unos, cantidad impar) → p=1 (bit de paridad par=1, ya que 3+1=4 es par) ✓. Paridad impar = `!p` = 0.

Ejercicio 6 — Suma en C2 de 16 bits con flags empaquetados

```
int sumaConFlags(short num1, short num2) {
    int sumaInt = (int)num1 + (int)num2;    // suma en 32 bits, sin truncar
    short sumaShort = num1 + num2;        // suma truncada a 16 bits (lo que "vería" el hardware)

    int Z = (sumaShort == 0) ? 1 : 0;
    int V = ((num1 >= 0 && num2 >= 0 && sumaShort < 0) ||
             (num1 < 0 && num2 < 0 && sumaShort >= 0)) ? 1 : 0;
    int C = ((sumaInt >> 16) & 1);    // ¿hubo acarreo más allá del bit 15?

    return ((sumaShort & 0xFFFF) << 3) | (C << 2) | (V << 1) | Z;
}
```

Metodología de los flags (igual que en el Práctico 1, Ejercicio 6):

- **Z:** el resultado (de 16 bits) es exactamente 0.
- **V (overflow con signo):** ambos operandos tienen el mismo signo y el resultado tiene signo distinto.
- **C (acarreo sin signo):** comparando la suma sin truncar (32 bits) contra la suma truncada, se ve si “se perdió” algo más allá del bit 15.

Ejemplo verificado: num1=25, num2=114. Suma=139 (cabe cómodo en 16 bits, sin overflow ni acarreo). Z=0 (139≠0), V=0 (ambos positivos, resultado positivo, sin overflow), C=0 (139 no necesita más de 16 bits).

Resultado = (139<<3)|0|0|0 = 1112 = **0b00000010001011000** (en los 19 bits del formato: bits 18..3 = 139, bits 2..0 = 000).

Ejercicio 7 — Rotación de bits

```
short rotar(short numero, int cantidad, char sentido) {
    unsigned short num = (unsigned short) numero; // sin signo, para que >> no arrastre
    el bit de signo
    cantidad = cantidad % 16;                      // rotar 16 veces = no rotar nada
    if (cantidad == 0) return numero;

    if (sentido == 0) { // derecha
        num = (num >> cantidad) | (num << (16 - cantidad));
    } else {           // izquierda
        num = (num << cantidad) | (num >> (16 - cantidad));
    }
    return (short) num; // el cast final descarta cualquier bit por encima de la
    posición 15
}
```

Por qué funciona: rotar es “desplazar y traer de vuelta por el otro extremo”. La parte que se “cae” de un lado (`num >> cantidad` para la derecha, que descarta los bits bajos) se recupera desplazando esos mismos bits hacia el extremo opuesto (`num << (16-cantidad)`), y el OR combina ambas mitades.

Ejemplo verificado: numero=0xACE1, cantidad=4, sentido=0 (derecha).

`num>>4` = 0x0ACE. `num<<12` (=16-4) trae los 4 bits bajos (0x1) al extremo alto: 0x1000. OR: 0x0ACE | 0x1000 = **0x1ACE** = **0001101011001110**.

Verificación manual: los últimos 4 bits de 0xACE1 son `0001`; moviéndolos al frente de los 12 bits restantes (`101011001110`) da `0001 101011001110` = 0x1ACE ✓ coincide.

Ejercicio 8 — Media precisión → simple precisión

Formato media precisión: 1 signo + 5 exponente (bias=15) + 10 mantisa. **Formato simple precisión:** 1 signo + 8 exponente (bias=127) + 23 mantisa.

Metodología: el signo se copia igual. El exponente hay que “desplazarlo” al nuevo bias: `nuevo-Exp = exp - 15 + 127 = exp + 112`. La mantisa se corre a la izquierda para ocupar las posiciones más altas de los 23 bits disponibles (rellenando con ceros a la derecha): `nuevaMant = mant << 13` (23-10=13 bits de relleno).

```
int mediaASimple(short num) {
    unsigned short unum = (unsigned short) num;
    int sign = (unum >> 15) & 0x1;
    int exp = (unum >> 10) & 0x1F;
    int mant = unum & 0x3FF;

    int nuevoExp, nuevaMant;

    if (exp == 0 && mant == 0) {                // cero (con signo)
        nuevoExp = 0; nuevaMant = 0;
    } else if (exp == 0x1F) {                    // infinito / NaN: se preserva
        nuevoExp = 0xFF; nuevaMant = mant << 13;
    } else {                                     // caso normal
        nuevoExp = exp - 15 + 127;
        nuevaMant = mant << 13;
    }

    return (sign << 31) | (nuevoExp << 23) | nuevaMant;
}
```

Ejemplo verificado (valor de referencia muy conocido): num=0x3C00 (1.0 en media precisión).

sign=0, exp=15 (biased; unbiased=0), mant=0.

nuevoExp = 15-15+127=127. nuevaMant=0.

Resultado = (127<<23) = **0x3F800000** — que es, efectivamente, la representación estándar de **1.0f** en simple precisión IEEE 754. ✓

Ejercicio 9 — Multiplicar dos floats sin usar `float`

Esta es la más compleja del práctico. **Idea general:** en punto flotante, multiplicar significa (1) multiplicar las mantisas (con el 1 implícito incluido) como enteros, (2) sumar los exponentes (restando el bias una vez, porque al sumar dos exponentes sesgados el bias queda contado dos veces), (3) normalizar el resultado (el producto de dos números en $[1,2)$ cae en $[1,4)$, puede necesitar 1 bit más de shift), y (4) el signo es el XOR de los signos.

```
int multiplicarPuntoFlotante(int num1, int num2) {
    int sign1 = (num1 >> 31) & 0x1;
    int sign2 = (num2 >> 31) & 0x1;
    int exp1  = (num1 >> 23) & 0xFF;
    int exp2  = (num2 >> 23) & 0xFF;
    long mant1 = (num1 & 0x7FFFFFFF) | 0x800000; // agrega el 1 implícito -> significando
de 24 bits
    long mant2 = (num2 & 0x7FFFFFFF) | 0x800000;

    int signRes = sign1 ^ sign2;

    // Caso especial: multiplicar por cero
    if ((exp1==0 && (num1&0x7FFFFFFF)==0) || (exp2==0 && (num2&0x7FFFFFFF)==0))
        return signRes << 31;

    long mantRes = mant1 * mant2;           // hasta 48 bits (24 bits x 24 bits)
    int  expRes  = exp1 + exp2 - 127;       // el bias estaba contado dos veces, se resta
una

    if (mantRes & (1L << 47)) {             // el producto de significandos cayó en
[2,4): un bit más arriba
        mantRes >>= 24;
        expRes += 1;
    } else {                                // cayó en [1,2): normal
        mantRes >>= 23;
    }

    int mantFinal = (int)(mantRes & 0x7FFFFFFF); // se descarta el 1 implícito

    return (signRes << 31) | (expRes << 23) | mantFinal;
}
```

Ejemplo verificado 1: $2.0 \times 3.0 = 6.0$

$2.0 = 0x40000000$ (exp=128, mant=0). $3.0 = 0x40400000$ (exp=128, mant=0x400000).

mant1=0x800000, mant2=0xC00000. mantRes=0x800000×0xC00000=1.5×2⁴⁶ (bit 47 **no** está prendido, cae en el caso normal).

$\text{mantRes} \gg 23 = 0xC00000$. $\text{mantFinal} = 0xC00000 \& 0x7FFFFFFF = 0x400000$.
 $\text{expRes} = 128 + 128 - 127 = 129$.

Resultado = $(129 \ll 23) | 0x400000 = \mathbf{0x40C00000}$, que decodifica a $\text{exp} = 129$ (unbiased=2),
 mantisa=0.5 $\rightarrow$ significando $1.5 \times 2^2 = \mathbf{6.0} \checkmark$

Ejemplo verificado 2: $1.5 \times 1.5 = 2.25$ (para probar la rama del “bit extra”)

$1.5 = 0x3FC00000$ ($\text{exp} = 127$, $\text{mant} = 0x400000$). $\text{mant1} = \text{mant2} = 0xC00000$.

$\text{mantRes} = 0xC00000 \times 0xC00000 = 2.25 \times 2^{46}$. Como $2.25 \geq 2$, el **bit 47 sí está prendido** $\rightarrow$ rama del shift extra.

$\text{mantRes} \gg 24 = 0x900000$. $\text{mantFinal} = 0x900000 \& 0x7FFFFFFF = 0x100000$.
 $\text{expRes} = 127 + 127 - 127 + 1 = 128$.

Resultado = $(128 \ll 23) | 0x100000 = \text{exp} = 128$ (unbiased=1), mantisa=0x100000 $\rightarrow$ fracción
 0.125 $\rightarrow$ significando $1.125 \times 2^1 = \mathbf{2.25} \checkmark$

Los dos ejemplos verificados de punta a punta (incluyendo la rama de normalización con bit extra) confirman que el algoritmo es correcto.